COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 10, 2026	**DATE SUBMITTED:**	September 15, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. __8.1__ **SORTING ALGORITHMS: SHELL, MERGE, AND QUICK SORT**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```cpp
void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";

        if ((i + 1) % 10 == 0) {
            std::cout << std::endl;
        }
    }

    std::cout << std::endl;
}


int main() {
    srand(time(0));
    const int SIZE = 100;

    int original[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    // Generates 100 random values for the array
    for (int i = 0; i < SIZE; i++) {
        original[i] = rand() % 1000 + 1;
    }

    // Copies the same array for each sorting algorithm to sort the same values
    for (int i = 0; i < SIZE; i++) {
        shellArray[i] = original[i];
        mergeArray[i] = original[i];
        quickArray[i] = original[i];
    }
```

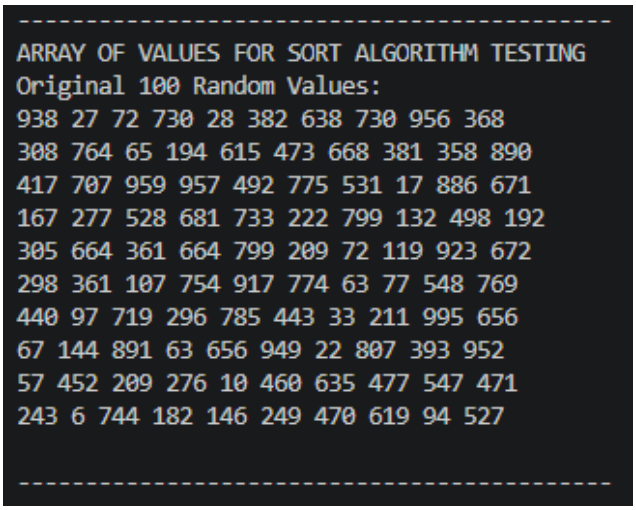
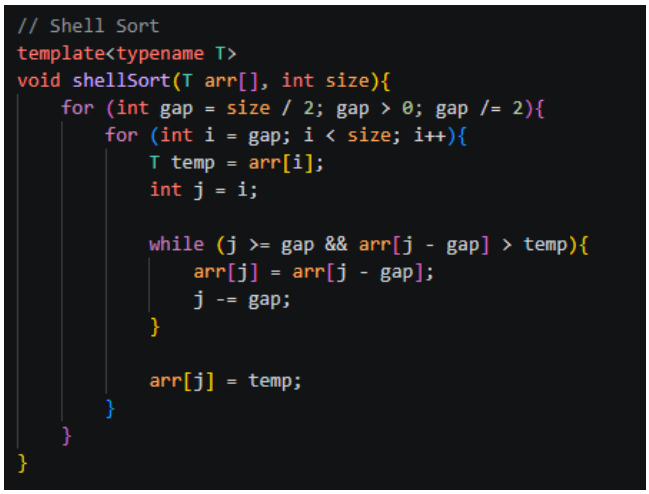
	
Observations	The code generates 100 random numbers from 1 to 1000 and stores them in the original array. The same values are then copied into three separate arrays for Shell Sort, Merge Sort, and Quick Sort. This is done so that all three sorting algorithms will work on the same set of numbers. The original array is displayed first to show the unsorted values before the sorting process begins.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```cpp
// Shell Sort
template<typename T>
void shellSort(T arr[], int size){
    for (int gap = size / 2; gap > 0; gap /= 2){
        for (int i = gap; i < size; i++){
            T temp = arr[i];
            int j = i;

            while (j >= gap && arr[j - gap] > temp){
                arr[j] = arr[j - gap];
                j -= gap;
            }

            arr[j] = temp;
        }
    }
}
```

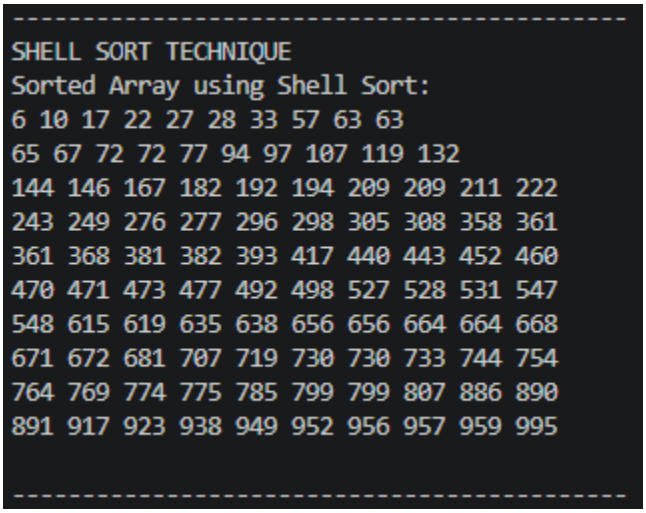
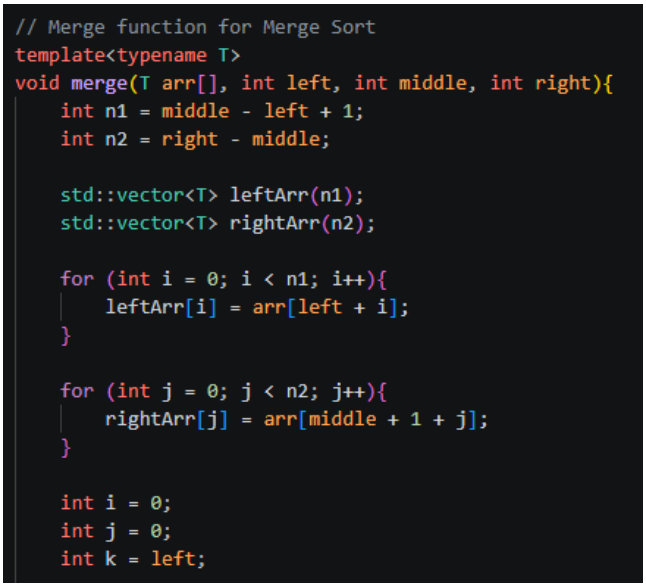
	
Observations	The Shell Sort code sorts the values from smallest to largest by comparing elements that are separated by a gap. The gap starts at half of the array size and becomes smaller after each round. When the gap becomes smaller, the elements are compared more closely until the gap reaches 1. At this point, the array is almost sorted, so the remaining elements can be arranged easily. The final result is a sorted array in ascending order.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
    while (i < n1 && j < n2){
        if (leftArr[i] <= rightArr[j]){
            arr[k] = leftArr[i];
            i++;
        }
        else{
            arr[k] = rightArr[j];
            j++;
        }

        k++;
    }
    while (i < n1){
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while (j < n2){
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
```

```
// Merge Sort
template<typename T>
void mergeSort(T arr[], int left, int right){
    if (left < right){
        int middle = left + (right - left) / 2;

        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);

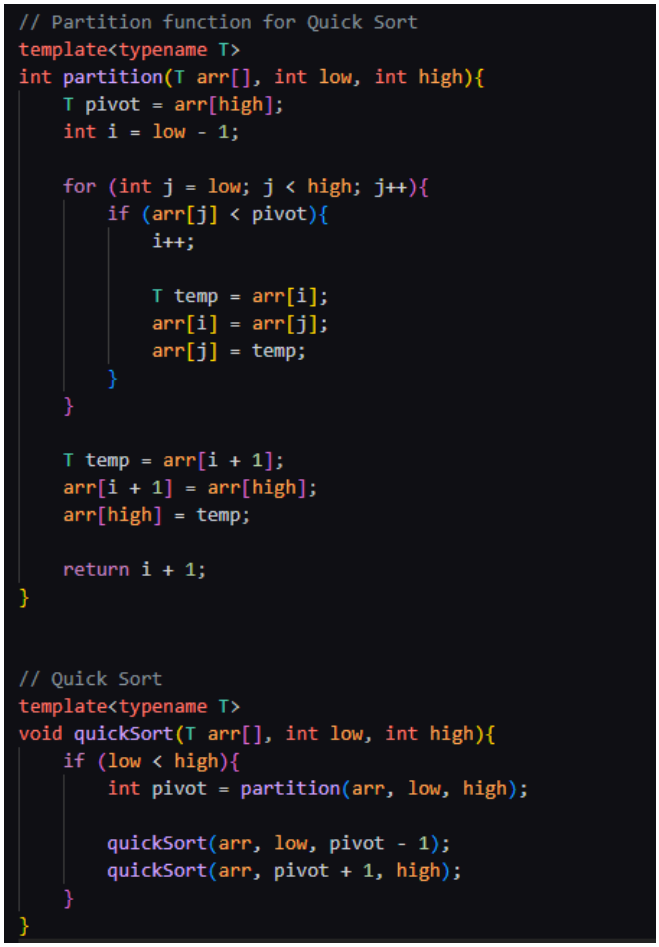
        merge(arr, left, middle, right);
    }
}
```

```
-----------------------------------------------
MERGE SORT ALGORITHM
Sorted Array using Merge Sort:
6 10 17 22 27 28 33 57 63 63
65 67 72 72 77 94 97 107 119 132
144 146 167 182 192 194 209 209 211 222
243 249 276 277 296 298 305 308 358 361
361 368 381 382 393 417 440 443 452 460
470 471 473 477 492 498 527 528 531 547
548 615 619 635 638 656 656 664 664 668
671 672 681 707 719 730 730 733 744 754
764 769 774 775 785 799 799 807 886 890
891 917 923 938 949 952 956 957 959 995

-----------------------------------------------
```

Observations	The Merge Sort code divides the array into smaller parts until each part contains only one element. It then compares and combines these smaller parts while arranging the values from smallest to largest. The process continues until all the parts are merged back into one complete sorted array. This makes the large set of 100 values easier to organize by sorting smaller sections first. The final output shows the values arranged in ascending order.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
// Partition function for Quick Sort
template<typename T>
int partition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++){
        if (arr[j] < pivot){
            i++;

            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}


// Quick Sort
template<typename T>
void quickSort(T arr[], int low, int high){
    if (low < high){
        int pivot = partition(arr, low, high);

        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}
```

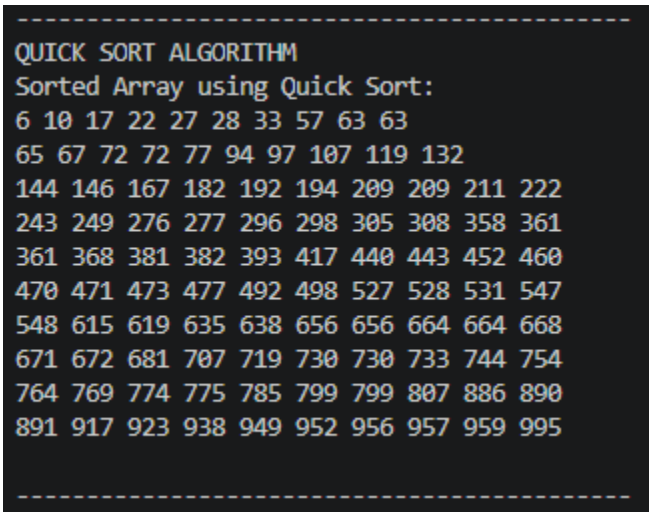
	
Observations	The Quick Sort code chooses a pivot value and compares the other elements with it. Values smaller than the pivot are moved to the left, while larger values are placed to the right. The same process is then repeated on the smaller sections of the array. This continues until all parts of the array are sorted. The final result is the 100 values arranged from smallest to largest.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer:

Yes. The left sublist and right sublist created after the partition process in Quick Sort, can also be sorted using other sorting algorithms, such as Shell Sort or Merge Sort.

For Example:

[34, 12, 45, 8, 23, 50,17]

Consider this array, suppose that Quick Sort uses 17 as its pivot. After Partitioning, the array can be divided into:

[34, 12, 45, 8, 23, 50,17]

Left Partition: [12, 8]
Pivot: [17]
Right Partition: [34, 45, 23, 50]

These partitions (instead of using quick sort again) can be sorted using Merge or Shell sort.

Left Partition: [12, 8] → [8, 12]
Right Partition: [34, 45, 23, 50] → [23, 34, 45, 50]

Therefore, yes, other sorting algorithms can be used to sort the left and right sublists after partitioning. The important thing is that the left sublist contains values smaller than the pivot, while the right sublist contains values greater than the pivot.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Answer:

For this array | {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74} | Quick sort can give the fastest time execution on average because it uses a “pivot” to divide the array into smaller subunits (sublists) and sort those

parts recursively. Quick Sort is fastest as it can just sort values without needing to create large additional copies of arrays.

Merge Sort and Quick Sort can have a time complexity of 0(N log N) because they divide the array into smaller parts repeatedly. The array is divided approximately log N times until the smallest sublist is reached. At each level of the process, about N elements are processed. Because there are approximately log N levels and N elements are handled at each level.

For Merge Sort, the array is divided into two halves, then the sorted halves are merged back together. For Quick Sort, the array is divided into parts based on a pivot, and the smaller parts are sorted recursively.

Therefore, both algorithms use a divide-and-conquer approach, which is why they can achieve O(N log N) time complexity. However, Quick Sort can become $O(N^2)$ in its worst case if poor pivot choices repeatedly create very uneven partitions.

Conclusion

This activity taught us how Shell Sort, Merge Sort, and Quick Sort are used to arrange data from the smallest value to the largest. We learned that each sorting algorithm uses a different method to organize the elements. Shell Sort compares values using gaps that become smaller, while Merge Sort divides the array into smaller parts and combines them in sorted order. Quick Sort uses a pivot to divide the array into smaller sections and sorts each section recursively. We also learned that Merge Sort and Quick Sort can achieve O(N log N) time complexity because they use a divide-and-conquer approach, although Quick Sort can become $O(N^2)$ when poor pivot choices create uneven partitions. Overall, this activity helped us understand how different sorting algorithms work and why choosing an appropriate algorithm is important when handling large amounts of data.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Only corrected my vocabulary and grammar.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.